Tutorial Sheet: Bubble Sort and Insertion Sort Timing Experiments in C on Ubuntu

Objective

This tutorial sheet explains how to:

- implement Bubble Sort and Insertion Sort in C on Ubuntu,
- measure execution time for different input patterns,
- compare standard Bubble Sort and optimized Bubble Sort,
- plot the measured values using GNUplot,
- analyze empirical behavior against theoretical complexity.

Tools Required

- Ubuntu Linux
- GCC compiler
- GNUplot
- text editor or IDE

Installation on Ubuntu

If GNUplot is not installed, use:

```
sudo apt update
sudo apt install build-essential gnuplot
```

To confirm installation:

```
gcc --version
gnuplot --version
```

General Experiment Method

Use the same workflow for all questions:

1. Choose several input sizes, for example $n = 100, 200, 400, 800, 1600$.
2. For each n, prepare three kinds of arrays:
    - best case,

- worst case,
- average case.

3. Run the sorting algorithm on each array multiple times if needed.
4. Measure only the sorting time, not file I/O or input generation.
5. Store the results in a text file, usually in two columns:

$$n \quad \text{time}$$

   or more columns if comparing multiple algorithms.

6. Use GNUplot to draw the graph.

Timing Advice

Use a high-resolution timer such as `clock_gettime()` or `gettimeofday()`. Avoid using simple `clock()` if you want finer measurements, especially for small input sizes.

A common timing pattern is:

- take timestamp before sorting,
- call the sorting function,
- take timestamp after sorting,
- compute elapsed time in seconds or milliseconds.

Input Generation Strategy

Best Case

- Bubble Sort standard: still performs many comparisons.
- Bubble Sort optimized: array already sorted, so the algorithm exits early.
- Insertion Sort: array already sorted, only one comparison per element.

Worst Case

- Bubble Sort: reverse-sorted array.
- Insertion Sort: reverse-sorted array.

Average Case

- Generate random data using `rand()`.
- Seed once using `srand(time(NULL))`.

Q2: Bubble Sort Analysis

What to implement

Create a standard Bubble Sort function in C. The inner loop compares adjacent elements and swaps them if needed. A standard version does not stop early even if the array becomes sorted.

What to measure

Record execution times for:

- best-case input,
- worst-case input,
- average-case input.

Expected behavior

- Best case: still close to $O(n^2)$ for standard Bubble Sort.
- Worst case: $O(n^2)$.
- Average case: $O(n^2)$.

Suggested output file format

```
# n best_time worst_time average_time
100 0.00012 0.00020 0.00015
200 0.00045 0.00082 0.00058
400 0.00180 0.00310 0.00220
```

GNUplot idea

Plot three curves on one graph:

- best case,
- worst case,
- average case.

Q3: Standard vs Optimized Bubble Sort

What to implement

Write two functions:

- standard Bubble Sort,
- optimized Bubble Sort with a swap flag.

Optimized idea

Set a flag at the beginning of each pass. If no swap happens during a full pass, stop immediately. This makes presorted input much faster.

What to measure

Use only best-case input, meaning already sorted arrays. Measure both versions for the same set of input sizes.

Expected behavior

- Standard Bubble Sort best case: still quadratic behavior.
- Optimized Bubble Sort best case: linear behavior, $O(n)$.

Suggested output file format

```
# n standard optimized
100 0.00011 0.00002
200 0.00042 0.00003
400 0.00170 0.00005
```

GNUplot idea

Plot the two timing columns against n, and clearly label them:

- standard bubble sort,
- optimized bubble sort.

Q4: Insertion Sort Analysis

What to implement

Create a standard Insertion Sort function in C. It builds a sorted prefix one element at a time by shifting larger elements to the right.

What to measure

Record execution times for:

- best-case input,
- worst-case input,
- average-case input.

Expected behavior

- Best case: $O(n)$, when the array is already sorted.
- Worst case: $O(n^2)$, when the array is reverse sorted.
- Average case: $O(n^2)$.

Suggested output file format

```
# n best_time worst_time average_time
100 0.00003 0.00019 0.00009
200 0.00005 0.00075 0.00030
400 0.00009 0.00300 0.00110
```

How to Plot with GNUplot

A typical GNUplot script can:

- set the title,
- label axes,
- choose line style and key position,
- plot each column from the file.

Example structure:

```
set title "Sorting Time Analysis"
set xlabel "Input Size (n)"
set ylabel "Time (seconds)"
set grid
set key left top
plot "data.dat" using 1:2 with linespoints title "Best Case", \
     "data.dat" using 1:3 with linespoints title "Worst Case", \
     "data.dat" using 1:4 with linespoints title "Average Case"
```

Compilation and Execution

Typical compilation command:

```
gcc program.c -o program
./program
```

If using `clock_gettime()`, compile with:

```
gcc program.c -o program -lrt
```

Common Mistakes

- Timing file generation together with sorting.
- Using too small n, which makes timing unreliable.
- Forgetting to regenerate the original array before each trial.
- Comparing different input sets instead of identical sizes.
- Not labeling graph axes clearly.

What to Write in the Analysis

For each question, include:

- the observed trend in the graph,
- the reason behind the trend,
- whether the experimental result matches the theory,
- a short conclusion about complexity.